Lecture Notes: TwinCAT 3

Test-Driven Development (TDD)

March 17, 2026

Contents

1 Introduction to Current Industry Practices **2**

2 What is Test-Driven Development (TDD)? **2**

2.1 The TDD Cycle: "Red-Green-Refactor" 2

3 Advantages of TDD **3**

4 Unit Testing with TcUnit **3**

4.1 Core Components of TcUnit 3

5 Practical Implementation: The Event Buffer **5**

5.1 Interface Design (The "What") 5

5.2 Refined Design through Testing 6

5.3 Writing the Test 9

6 Execution and Results **13**

7 Integration with Continuous Integration (CI) **13**

Quick Reference Card **14**

1 Introduction to Current Industry Practices

- **The Problem:** In industrial automation, code is often deployed to live systems (costing millions of euros) with a "let's see if it works" attitude and very little automated testing.
- **The Opportunity:** Adopting practices from traditional software development, such as **Test-Driven Development (TDD)**, can vastly increase code quality and reliability.

Approach	Traditional PLC	TDD Approach
Testing	Manual, on real hardware	Automated, simulated
Bug Discovery	During commissioning	During development
Code Changes	High risk, avoided	Low risk, embraced
Documentation	Often outdated	Tests are living docs

Table 1: Traditional vs. TDD Development Approaches

⚠ **Warning:** Deploying untested code to production machinery can result in equipment damage, production losses, and safety incidents. Automated testing significantly reduces these risks.

2 What is Test-Driven Development (TDD)?

TDD is the practice of **writing tests before writing the implementation code**. Instead of focusing on *how* to develop, TDD forces the developer to first define *what* the code should do.

2.1 The TDD Cycle: "Red-Green-Refactor"

1. **Red Phase:** Write a test for a specific behavior. Because no implementation code exists yet, the test will **fail**.
2. **Green Phase:** Write just enough implementation code to make the test pass. The focus is strictly on meeting the test criteria, not on making the code "pretty".
3. **Refactor Phase:** Once the tests pass, improve the code's design, rename variables, or optimize algorithms. The tests act as a safety net to ensure behavior remains unchanged.

RED Write failing test	→	**GREEN** Write minimal code
↑		↓
REFACTOR Improve code quality		

Figure 1: The TDD Red-Green-Refactor Cycle

✖ **Red Phase:** In the Red Phase, you write a test that describes the desired behavior. The test **must fail** initially—this confirms that the test is actually testing something meaningful and that the feature doesn't already exist.

Green Phase: In the Green Phase, write the minimum amount of code necessary to make the test pass. Don't over-engineer or add extra features—just satisfy the test requirements.

3 Advantages of TDD

- **Regression Testing:** A suite of tests allows developers to change or add functionality at any time, with the confidence that existing features haven't been broken.
- **Modular Design:** TDD naturally leads to smaller, more flexible, and more modular function blocks.
- **Clear Documentation:** Test cases serve as living documentation and acceptance criteria, showing exactly what a function block should provide given specific inputs.
- **Lower Bug Counts:** Writing tests first encourages developers to consider edge cases they might otherwise ignore.

Benefit	Impact on PLC Development
Regression Testing	Safe to modify code during commissioning
Modular Design	Easier to reuse FBs across projects
Documentation	Tests explain expected behavior
Edge Case Coverage	Fewer surprises with unusual inputs
Faster Debugging	Pinpoint failures immediately

Table 2: TDD Benefits for Industrial Automation

Tip: Tests serve as executable documentation. When a new developer joins the project, they can read the tests to understand exactly what each function block is supposed to do.

4 Unit Testing with TcUnit

Unit testing involves validating that individual components of software perform as expected. For TwinCAT 3, the preferred tool is **TcUnit**, an open-source framework licensed under the MIT license.

4.1 Core Components of TcUnit

- **Test Suite:** A collection of related tests, defined as a function block that extends `FB_TestSuite`.
- **Test Runner:** Collects and presents results via the `TcUnit.RUN()` command.
- **Assertions:** Methods used to verify the state of a unit under test (e.g., `AssertTrue`, `AssertFalse`, `AssertEquals`).

Assertion	Usage	Description
AssertTrue	AssertTrue(bCondition, 'msg')	Passes if condition is TRUE
AssertFalse	AssertFalse(bCondition, 'msg')	Passes if condition is FALSE
AssertEquals	AssertEquals(exp, act, 'msg')	Passes if values are equal
AssertEquals_STRING	AssertEquals_STRING(exp, act, 'msg')	String comparison
AssertEquals_INT	AssertEquals_INT(exp, act, 'msg')	Integer comparison
AssertEquals_REAL	AssertEquals_REAL(exp, act, delta, 'msg')	Real with tolerance

Table 3: Common TcUnit Assertion Methods

TcUnit Test Runner

```
// ====================================================
// PROGRAM: PRG_TEST
// Main test runner program for TcUnit
// This program should be called from a separate test task
// ====================================================

PROGRAM PRG_TEST
VAR
    // ====================================================
    // TEST SUITE INSTANCES
    // Each test suite is a function block that extends FB_TestSuite
    // Instantiate all test suites you want to run
    // ====================================================

    // Test suite for the Event Buffer functionality
    fbEventBufferTests      : FB_EventBufferTests;

    // Test suite for sensor handling
    fbSensorHandlerTests    : FB_SensorHandlerTests;

    // Test suite for motor controller
    fbMotorControllerTests  : FB_MotorControllerTests;

    // Test suite for string utilities
    fbStringUtilsTests      : FB_StringUtilsTests;

    // Test suite for data validation
    fbDataValidationTests   : FB_DataValidationTests;

END_VAR

// ====================================================
// CALL TEST SUITES
// Each test suite must be called to execute its tests
// TcUnit automatically discovers and runs all TEST() methods
// ====================================================

// Call each test suite - order doesn't matter
fbEventBufferTests();
fbSensorHandlerTests();
fbMotorControllerTests();
fbStringUtilsTests();
fbDataValidationTests();

```

```
// ====================================================
// RUN TCUNIT
// This MUST be called after all test suites
// It collects results and outputs them to the error list
// ====================================================
TcUnit.RUN();
```

```
// ====================================================
// ALTERNATIVE: Minimal Test Runner
// For projects with just one or two test suites
// ====================================================

PROGRAM PRG_TEST_MINIMAL
VAR
    // Single test suite instance
    fbMyTests : FB_MyFeatureTests;
END_VAR

// Call the test suite
fbMyTests();

// Run TcUnit to collect and report results
TcUnit.RUN();

// ====================================================
// TASK CONFIGURATION NOTE:
// ====================================================
// Create a separate task for testing with:
// - Lower priority than production tasks
// - Cycle time: 10ms or slower is fine
// - Only run during development/testing phases
//
// Task: Test_Task
//   Priority: 25 (lower than main task)
//   Cycle Time: 10 ms
//   Program: PRG_TEST
// ====================================================
```

Note: TcUnit results appear in the Visual Studio Error List window. Filter by "TcUnit" to see only test-related messages. Green checkmarks indicate passing tests; red X marks indicate failures.

5 Practical Implementation: The Event Buffer

The sources demonstrate TDD by building an **Event Buffer** to solve a limitation in a CSV logger that could only handle one event at a time.

5.1 Interface Design (The "What")

Before implementation, the interface for the `FB_EventBuffer` is defined with methods such as `AddEvent`, `GetAndRemoveEvent`, and `GetNumberOfEvents`.

5.2 Refined Design through Testing

During the testing phase, it was realized that hard-coding a buffer size was too restrictive. The design was updated to use a **constructor (FB_init)**, allowing the user to provide an external memory buffer of any size.

FB_init Constructor for Buffer

```
// ====================================================
// FUNCTION BLOCK: FB_EventBuffer
// A FIFO buffer for storing events with configurable size
// Uses FB_init constructor for flexible memory allocation
// ====================================================

FUNCTION_BLOCK FB_EventBuffer
VAR_INPUT
END_VAR

VAR_OUTPUT
    bError          : BOOL;         // Error flag
    nErrorID        : UDINT;        // Error code
END_VAR

VAR
    // ====================================================
    // INTERNAL BUFFER MANAGEMENT
    // These are set by FB_init and used internally
    // ====================================================
    pBuffer         : PVOID;        // Pointer to external buffer memory
    nBufferSize     : UDINT;        // Size of buffer in bytes
    nEventSize      : UDINT;        // Size of one event structure
    nMaxEvents      : UDINT;        // Maximum number of events

    // FIFO management
    nHead           : UDINT := 0;   // Read position (oldest event)
    nTail           : UDINT := 0;   // Write position (next free slot)
    nCount          : UDINT := 0;   // Current number of events

    bInitialized    : BOOL := FALSE;
END_VAR
```

```
// ====================================================
// METHOD: FB_init (Constructor)
// Called automatically when the FB is instantiated
// Allows user to provide external buffer memory
// ====================================================

METHOD FB_init : BOOL
VAR_INPUT
    bInitRetains    : BOOL;     // TRUE if retain variables initialized
    bInCopyCode     : BOOL;     // TRUE if called from copy code

    // ====================================================
    // USER-PROVIDED BUFFER PARAMETERS
```

```
    // The user must allocate an array of bytes and pass:
    // - pBufferMemory: Pointer to the array (use ADR())
    // - nBufferMemorySize: Size in bytes (use SIZEOF())
    // ====================================================
    pBufferMemory       : PVOID;    // Pointer to user's byte array
    nBufferMemorySize   : UDINT;    // Size of the byte array

END_VAR

// ====================================================
// INITIALIZATION LOGIC
// Store the buffer reference and calculate capacity
// ====================================================

// Validate inputs
IF pBufferMemory = 0 OR nBufferMemorySize = 0 THEN
    bError := TRUE;
    nErrorID := 16#8001;    // Invalid parameters
    bInitialized := FALSE;
    FB_init := FALSE;
    RETURN;
END_IF

// Store buffer reference
pBuffer := pBufferMemory;
nBufferSize := nBufferMemorySize;

// Calculate event size and maximum capacity
nEventSize := SIZEOF(ST_Event);
nMaxEvents := nBufferSize / nEventSize;

// Validate we can store at least one event
IF nMaxEvents < 1 THEN
    bError := TRUE;
    nErrorID := 16#8002;    // Buffer too small
    bInitialized := FALSE;
    FB_init := FALSE;
    RETURN;
END_IF

// Clear the buffer
MEMSET(destAddr := pBuffer, fillByte := 0, n := nBufferSize);

// Reset FIFO pointers
nHead := 0;
nTail := 0;
nCount := 0;

// Mark as successfully initialized
bInitialized := TRUE;
bError := FALSE;
nErrorID := 0;
FB_init := TRUE;
```

```
// ====================================================
// USAGE EXAMPLE: Instantiating FB_EventBuffer
// ====================================================

```

```
PROGRAM MAIN
VAR
    // ====================================================
    // USER-ALLOCATED BUFFER MEMORY
    // Size determines how many events can be stored
    // Calculate: nMaxEvents = SIZEOF(aEventMemory) / SIZEOF(ST_Event)
    // ====================================================

    // Buffer for approximately 100 events
    // (Adjust size based on ST_Event structure size)
    aEventMemory     : ARRAY[0..9999] OF BYTE;

    // ====================================================
    // EVENT BUFFER INSTANCE WITH FB_init
    // The constructor parameters are passed at declaration
    // ====================================================
    fbEventBuffer    : FB_EventBuffer(
                         pBufferMemory := ADR(aEventMemory),
                         nBufferMemorySize := SIZEOF(aEventMemory)
                       );

    // Alternative: Smaller buffer for testing
    aSmallBuffer     : ARRAY[0..999] OF BYTE;
    fbSmallEventBuffer : FB_EventBuffer(
                         pBufferMemory := ADR(aSmallBuffer),
                         nBufferMemorySize := SIZEOF(aSmallBuffer)
                       );

    // Test events
    stNewEvent       : ST_Event;
    stReadEvent      : ST_Event;
    bAddSuccess      : BOOL;
    bReadSuccess     : BOOL;
    nEventCount      : UDINT;

END_VAR

// Check if initialization was successful
IF fbEventBuffer.bError THEN
    // Handle initialization error
    // nErrorID contains the error code
END_IF

// Add an event
stNewEvent.nEventID := 1001;
stNewEvent.sDescription := 'Test event';
stNewEvent.tTimestamp := DT#2024-01-15-10:30:00;
bAddSuccess := fbEventBuffer.AddEvent(stEvent := stNewEvent);

// Get number of events
nEventCount := fbEventBuffer.GetNumberOfEvents();

// Read and remove oldest event
bReadSuccess := fbEventBuffer.GetAndRemoveEvent(stEvent =>
    stReadEvent);
```

Tip: Using `FB_init` with external buffer memory allows the same function block to be used with different buffer sizes across projects. The caller controls memory allocation, making the FB more flexible and testable.

5.3 Writing the Test

A test suite is created to verify that if three events are added, they can be read back in the same order (**First In First Out / FIFO**) and that the data (identity, text, timestamp) remains accurate.

Writing a Unit Test

```
// ====================================================
// FUNCTION BLOCK: FB_EventBufferTests
// Test suite for FB_EventBuffer
// Extends FB_TestSuite from TcUnit library
// ====================================================

FUNCTION_BLOCK FB_EventBufferTests EXTENDS FB_TestSuite
VAR
    // ====================================================
    // TEST FIXTURES
    // Shared resources used across multiple tests
    // ====================================================

    // Buffer memory for testing
    aTestBuffer      : ARRAY[0..4999] OF BYTE;

    // Event buffer instance under test
    fbBuffer         : FB_EventBuffer(
                         pBufferMemory := ADR(aTestBuffer),
                         nBufferMemorySize := SIZEOF(aTestBuffer)
                       );

    // Test events
    stEvent1         : ST_Event;
    stEvent2         : ST_Event;
    stEvent3         : ST_Event;
    stReadEvent      : ST_Event;

    // Test state management
    nTestStep        : INT := 0;
    bTestComplete    : BOOL := FALSE;

END_VAR
```

```
// ====================================================
// METHOD: Test_AddEvent_ReturnsTrue
// Verifies that AddEvent returns TRUE on success
// ====================================================

METHOD Test_AddEvent_ReturnsTrue
VAR
    bResult          : BOOL;
    stTestEvent      : ST_Event;
```

```
END_VAR

// ====================================================
// TEST() marks the beginning of a test case
// The string parameter is the test name shown in results
// ====================================================
TEST('AddEvent should return TRUE when buffer has space');

// Arrange: Prepare test event
stTestEvent.nEventID := 100;
stTestEvent.sDescription := 'Test event for add';
stTestEvent.tTimestamp := DT#2024-01-15-12:00:00;

// Act: Call the method under test
bResult := fbBuffer.AddEvent(stEvent := stTestEvent);

// Assert: Verify the expected outcome
AssertTrue(
    Condition := bResult,
    Message := 'AddEvent should return TRUE when buffer is not full'
);

// ====================================================
// TEST_FINISHED() marks the end of the test case
// MUST be called to complete the test
// ====================================================
TEST_FINISHED();
```

```
// ====================================================
// METHOD: Test_FIFO_Order
// Verifies that events are returned in FIFO order
// ====================================================

METHOD Test_FIFO_Order
VAR
    bAdd1, bAdd2, bAdd3 : BOOL;
    bRead1, bRead2, bRead3 : BOOL;
    stRead1, stRead2, stRead3 : ST_Event;
END_VAR

TEST('Events should be retrieved in FIFO order');

// Arrange: Create three distinct events
stEvent1.nEventID := 1;
stEvent1.sDescription := 'First event';
stEvent1.tTimestamp := DT#2024-01-15-10:00:00;

stEvent2.nEventID := 2;
stEvent2.sDescription := 'Second event';
stEvent2.tTimestamp := DT#2024-01-15-10:01:00;

stEvent3.nEventID := 3;
stEvent3.sDescription := 'Third event';
stEvent3.tTimestamp := DT#2024-01-15-10:02:00;

// Act: Add three events in order
bAdd1 := fbBuffer.AddEvent(stEvent := stEvent1);
bAdd2 := fbBuffer.AddEvent(stEvent := stEvent2);
```

```
bAdd3 := fbBuffer.AddEvent(stEvent := stEvent3);

// Act: Read events back
bRead1 := fbBuffer.GetAndRemoveEvent(stEvent => stRead1);
bRead2 := fbBuffer.GetAndRemoveEvent(stEvent => stRead2);
bRead3 := fbBuffer.GetAndRemoveEvent(stEvent => stRead3);

// Assert: All adds succeeded
AssertTrue(bAdd1 AND bAdd2 AND bAdd3,
    'All AddEvent calls should succeed');

// Assert: All reads succeeded
AssertTrue(bRead1 AND bRead2 AND bRead3,
    'All GetAndRemoveEvent calls should succeed');

// Assert: Events returned in correct FIFO order
AssertEquals_DINT(
    Expected := 1,
    Actual := stRead1.nEventID,
    Message := 'First read should return event ID 1'
);

AssertEquals_DINT(
    Expected := 2,
    Actual := stRead2.nEventID,
    Message := 'Second read should return event ID 2'
);

AssertEquals_DINT(
    Expected := 3,
    Actual := stRead3.nEventID,
    Message := 'Third read should return event ID 3'
);

// Assert: Event data integrity
AssertEquals_STRING(
    Expected := 'First event',
    Actual := stRead1.sDescription,
    Message := 'First event description should match'
);

TEST_FINISHED();
```

```
// ====================================================
// METHOD: Test_GetNumberOfEvents
// Verifies the count of events in buffer
// ====================================================

METHOD Test_GetNumberOfEvents
VAR
    nCount      : UDINT;
    stTestEvent : ST_Event;
END_VAR

TEST('GetNumberOfEvents should return correct count');

// Arrange: Start with empty buffer (after reset)
stTestEvent.nEventID := 999;
```

```
stTestEvent.sDescription := 'Count test';

// Assert: Initially empty
nCount := fbBuffer.GetNumberOfEvents();
AssertEquals_UDINT(
    Expected := 0,
    Actual := nCount,
    Message := 'Empty buffer should have 0 events'
);

// Act: Add one event
fbBuffer.AddEvent(stEvent := stTestEvent);

// Assert: Count is 1
nCount := fbBuffer.GetNumberOfEvents();
AssertEquals_UDINT(
    Expected := 1,
    Actual := nCount,
    Message := 'Buffer should have 1 event after adding'
);

// Act: Add another event
fbBuffer.AddEvent(stEvent := stTestEvent);

// Assert: Count is 2
nCount := fbBuffer.GetNumberOfEvents();
AssertEquals_UDINT(
    Expected := 2,
    Actual := nCount,
    Message := 'Buffer should have 2 events after adding second'
);

TEST_FINISHED();
```

```
// ====================================================
// METHOD: Test_BufferFull_ReturnsFalse
// Verifies behavior when buffer is full
// ====================================================

METHOD Test_BufferFull_ReturnsFalse
VAR
    i            : INT;
    bResult      : BOOL;
    stTestEvent  : ST_Event;
    nMaxEvents   : UDINT;
END_VAR

TEST('AddEvent should return FALSE when buffer is full');

// Arrange: Fill the buffer to capacity
nMaxEvents := SIZEOF(aTestBuffer) / SIZEOF(ST_Event);
stTestEvent.nEventID := 0;
stTestEvent.sDescription := 'Fill event';

FOR i := 1 TO UDINT_TO_INT(nMaxEvents) DO
    stTestEvent.nEventID := i;
    fbBuffer.AddEvent(stEvent := stTestEvent);
END_FOR
```

```
// Act: Try to add one more event
stTestEvent.nEventID := 9999;
bResult := fbBuffer.AddEvent(stEvent := stTestEvent);

// Assert: Should return FALSE (buffer full)
AssertFalse(
    Condition := bResult,
    Message := 'AddEvent should return FALSE when buffer is full'
);

TEST_FINISHED();
```

Warning: Each test should be independent and not rely on the state left by previous tests. Use setup and teardown logic if needed, or create fresh instances of the unit under test for each test method.

6 Execution and Results

- **Environment:** Tests can be executed locally within the TwinCAT development environment.
- **Reporting:** Results are displayed in the **Error List** of Visual Studio or TcXaeShell.
- **Failure Analysis:** If an assertion fails, TcUnit provides the name of the test, the expected value, and the actual value to help locate the bug.

Result	Icon	Meaning	Action
PASS	✅	Test assertion succeeded	None required
FAIL	❌	Test assertion failed	Fix code or test
ERROR	⚠	Test threw exception	Debug test setup

Table 4: TcUnit Test Result Indicators

Test Name	Result	Message
AddEvent should return TRUE...	PASS	—
Events should be retrieved in FIFO...	PASS	—
GetNumberOfEvents should return...	FAIL	Expected: 2, Actual: 1
AddEvent should return FALSE...	PASS	—

Figure 2: Example TcUnit Error List Output

Note: When a test fails, TcUnit displays: (1) the test name, (2) the expected value, (3) the actual value, and (4) your custom message. This information helps quickly identify the root cause of the failure.

7 Integration with Continuous Integration (CI)

TDD is often a component of **Continuous Integration (CI)**, where tests are executed automatically every time a developer commits code to version control. This ensures that the "shared

mainline" of code remains stable.

CI Tool	TwinCAT Support	Notes
Jenkins	Via TcUnit scripts	Most common in industry
Azure DevOps	Via build agents	Good for Microsoft shops
GitLab CI	Via Docker/scripts	Open source option
GitHub Actions	Via custom runners	Modern, cloud-based

Table 5: CI Tools Compatible with TcUnit

Developer Writes code	→	**Git Push** Commits changes	→	**CI Server** Runs tests
		↓		
Tests Pass		→		Deploy/Merge
Tests Fail		→		Notify Developer

Figure 3: Continuous Integration Workflow with TcUnit

Tip: For CI integration, TcUnit can output results in xUnit XML format, which is compatible with most CI tools. Configure your CI pipeline to parse these results and fail the build if any tests fail.

Quick Reference Card

TDD Cycle

1. **RED**: Write a failing test
2. **GREEN**: Write minimal code to pass
3. **REFACTOR**: Improve code quality
4. Repeat

TcUnit Test Structure

```
// Test Suite (extends FB_TestSuite)
FUNCTION_BLOCK FB_MyTests EXTENDS FB_TestSuite

// Test Method
METHOD Test_MyFeature
    TEST('Description of what is being tested');

    // Arrange - set up test conditions
    // Act - call the code under test
    // Assert - verify results
    AssertTrue(bCondition, 'Error message');

    TEST_FINISHED();
END_METHOD
```

Common Assertions

```
AssertTrue(bCondition, 'message');
AssertFalse(bCondition, 'message');
AssertEquals(nExpected, nActual, 'message');
AssertEquals_STRING(sExpected, sActual, 'message');
AssertEquals_INT(nExpected, nActual, 'message');
AssertEquals_REAL(fExpected, fActual, fDelta, 'message');
```

Test Runner

```
PROGRAM PRG_TEST
VAR
    fbTests : FB_MyTestSuite;
END_VAR

fbTests();
TcUnit.RUN();
```

Last Updated: March 17, 2026